Low-Level Design (LLD) — Chat Backend

This document describes the **implemented** behavior of the Node.js/Express chat backend: HTTP API contracts, persistence schema, real-time layer, middleware, validation, and cross-cutting concerns. It is derived from the `src/` tree and is intended for engineers maintaining or extending the service.

---

1. Purpose and scope

Item	Description
Product	Multi-user chat with **groups**, **direct messages**, **read receipts**, **in-app notifications**, and **presence** (online counts / user online flag).
Auth model	Header-based **acting user** identity: `x-user-id` (numeric). Not a cryptographic identity; suitable for demos or trusted networks only.

---

2. Technology stack

Layer	Technology
Runtime	Node.js (CommonJS `require` / `module.exports` via TypeScript compile)
HTTP	Express **5.x**
Real-time	Socket.IO **4.x**
ORM / DB	Sequelize **6.x** / **MySQL** (mysql2)
Validation	Joi **18.x**
Security headers	Helmet
CORS	`cors` (configured `origin: "*"`)

Entry: `src/server.ts` (bootstraps DB, HTTP server, Socket.IO, graceful shutdown).

---

3. Process architecture

3.1 `src/app.ts` — HTTP application

Responsibilities:

1. **CORS** — allow all origins (`*`).
2. **Helmet** — default security-related HTTP headers.
3. **`express.json()`** — parse JSON bodies (UTF-8).
4. Mount **`/api/v1`** → consolidated API router (`src/routes/index.ts`).
5. **404** — `notFoundMiddleware` for unknown routes under this app.
6. **Error** — `errorMiddleware` as the final error handler.

No database or Socket.IO wiring in `app.ts`; those live in `server.ts`.

3.2 `src/server.ts` — composition root

Sequence:

1. Load Express request type augmentation (`./types/express-request-augmentation`).
2. **`connectDB()`** — Sequelize `authenticate()`.
3. **`sequelize.sync()`** — in **development** (`NODE_ENV === "development"`), uses `alter: true`; otherwise sync without alter. **Logging disabled** for sync.
4. **`http.createServer(app)`** — single HTTP server for REST + WebSocket upgrade.
5. **Socket.IO** — attach to HTTP server; CORS `origin: "*"`; methods `GET`, `POST`, `PUT`, `DELETE`.
6. **`setSocketServer(socketServer)`** — assigns module-level `io` in `src/socket/io.ts` for use from controllers.
7. **`registerSocketEvents(socketServer)`** — connection lifecycle and chat events.
8. **`httpServer.listen(env.port)`** — default port from env (see §4).
9. **Graceful shutdown** on `SIGTERM` / `SIGINT`: disconnect sockets → close Socket.IO → `sequelize.close()` → `httpServer.close()` → `process.exit(0)`.

`EADDRINUSE` logs a hint to free the port or set `PORT`.

---

4. Configuration (`src/config`)

4.1 `env.ts` and project `.env`

The checked-in local **`.env`** (first seven lines) for this workspace is:

```
PORT=5000
NODE_ENV=development
DB_HOST=localhost
DB_PORT=3306
DB_NAME=msg
DB_USER=root
DB_PASSWORD=mysql
```

4.2 `database.ts`

- Single **`Sequelize`** instance: dialect `mysql`, credentials from `env.db`.
- **`logging`**: `console.log` in development only; otherwise `false`.
- **`connectDB`**: `sequelize.authenticate()`.

---

5. Standard API envelope (`src/utils/apiResponse.ts`)

All JSON responses use a common shape:

Success

```
{
  "success": true,
  "message": "Human-readable summary",
  "data": {}
}
```

Error

```
{
  "success": false,
  "message": "Primary error message",
  "errors": null
}
```

Validation failures set **`errors`** to an array of Joi detail strings (see §8.2).

---

6. HTTP errors (`src/utils/httpError.ts`)

`HttpError` extends `Error` with **`statusCode`**. Services/controllers throw `HttpError` for predictable client errors (400, 403, 404). The **`errorMiddleware`** maps:

- **`HttpError`** → `statusCode` from instance.
- **Other errors** → status **500** unless `err.statusCode` is set.
- **Message exposure**: if the status code is less than 500 **or** `NODE_ENV !== "production"`, the client sees `err.message`; in production for 500 or above, message is generic `"Internal server error"`.

---

7. Request lifecycle helpers

7.1 `asyncHandler` (`src/utils/asyncHandler.ts`)

Wraps async route handlers so rejected promises call **`next(err)`**, integrating with `errorMiddleware`.

7.2 Express augmentation (`src/types/express-request-augmentation.ts`)

Field	Type	Set by
`req.auth_user_id`	`number`	`requireUserContext`
`req.validated`	`unknown`	`validateBody` (Joi result)
`req.request_id`	`string`	Reserved; not set in current code

---

8. Middleware

8.1 `requireUserContext` (`src/middlewares/auth-context.middleware.ts`)

- Reads `x-user-id` header.
- Parses as number; rejects if missing, NaN, or ≤ 0 → **401** with `errorResponse("Missing or invalid x-user-id header")`.
- Sets `req.auth_user_id`.

8.2 `validateBody(schema)` (`src/middlewares/validate.middleware.ts`)

- Joi `schema.validate(req.body, { abortEarly: false, stripUnknown: true })`.
- On failure → **400**, `errorResponse("Validation failed", [detail messages])`.
- On success → `req.validated = value`, `next()`.

8.3 `notFoundMiddleware`

404 — `errorResponse("Route not found: " + req.originalUrl)`.

8.4 `errorMiddleware`

Final error handler; always JSON (see §6).

---

9. Routing topology (`src/routes/index.ts`)

Base path: `/api/v1`.

Mount order on `apiRouter`:

1. `/users` → `user.routes.ts` (**no** global `requireUserContext` before this mount for paths under `/users` handled entirely inside `userRouter`).
2. `requireUserContext` — applies to subsequent mounts (`/groups`, `/chat`, `/notifications`).
3. `/groups`, `/chat`, `/notifications`.

Important: `POST /users` and `GET /users` are reachable **without** `x-user-id`. `GET /users/directory` adds `requireUserContext` on the route itself (directory requires identity). All `/groups/*`, `/chat/*`, `/notifications/*` require `x-user-id` via step 2.

---

10. REST API — full catalog

Unless noted, assume:

- `Content-Type: application/json`
- **Success** uses the envelope in §5 with `success: true`.
- **Numeric path params** (`:group_id`, etc.) are parsed with `Number()` in controllers; invalid numbers become `NaN` and may yield unexpected DB behavior or empty results — clients should send valid integers.

10.1 Users (`/api/v1/users`)

`POST /api/v1/users` **— Create user**

Aspect	Detail
Auth	None
Body validation	`createUserSchema` (Joi)

Request body

```
{
  "first_name": "Ada",
  "last_name": "Lovelace",
  "email_id": "ada@example.com"
}
```

Rules: `first_name` string length 2–100; `last_name` 1–100; `email_id` valid email.

Success — 201

```
{
  "success": true,
  "message": "User created successfully",
  "data": {
    "user_id": 1,
    "first_name": "Ada",
    "last_name": "Lovelace",
    "email_id": "ada@example.com",
    "is_active": true,
    "created_at": "2026-05-11T12:00:00.000Z",
    "updated_at": "2026-05-11T12:00:00.000Z"
  }
}
```

Errors

- **400** — validation failed (Joi messages in `errors`).
- **500** — DB/unique constraint (e.g. duplicate `email_id`); in production, generic message for 500.

`GET /api/v1/users` — List all users

Aspect	Detail
Auth	None

Success — 200

```
{
  "success": true,
```

```
  "message": "Users fetched successfully",
  "data": [
    {
      "user_id": 2,
      "first_name": "Alan",
      "last_name": "Turing",
      "email_id": "alan@example.com",
      "is_active": true,
      "created_at": "...",
      "updated_at": "..."
    }
  ]
}
```

Ordering: `user_id` **DESC**.

`GET /api/v1/users/directory` **— Paginated user directory (for DMs)**

Aspect	Detail
Auth	`x-user-id` (required on this route)
Query	`page` (default 1, min 1), `limit` (default 10, max 50)

Request headers

```
x-user-id: 1
```

Example: `GET /api/v1/users/directory?page=1&limit=10`

Success — 200

```
{
  "success": true,
  "message": "User directory fetched successfully",
  "data": {
    "items": [
      {
        "user_id": 2,
        "first_name": "Bob",
        "last_name": "Builder",
        "email_id": "bob@example.com",
        "is_active": true,
        "is_online": false
      }
    ],
    "total": 42,
    "page": 1,
```

```
      "limit": 10,
      "total_pages": 5
    }
  }
```

Notes

- Excludes the requesting user (`x-user-id`) from results.
- **`is_online`** from in-memory **`presenceService`** (Socket.IO connection state), not DB.

Errors

- **401** — missing/invalid `x-user-id`.

10.2 Groups (`/api/v1/groups`)

All routes require **`x-user-id`** (global middleware after `/users`).

`GET /api/v1/groups/my` — List my groups

Success — 200

```
{
  "success": true,
  "message": "Groups fetched successfully",
  "data": [
    {
      "group_id": 10,
      "group_title": "Engineering",
      "group_type": "group",
      "admin_user_id": 1,
      "is_active": true,
      "created_at": "...",
      "updated_at": "..."
    }
  ]
}
```

Logic: `GroupUserMap` rows for caller with **`is_exited: false`**, then **`ChatGroup`** where **`is_active: true`**.

`POST /api/v1/groups` — Create group

Body validation: `createGroupSchema`.

Request body

```
{
  "group_title": "Engineering",
  "user_ids": [2, 3]
}
```

Rules: `group_title` 2–120 chars; `user_ids` array of positive integers (default `[]`).

Success — 201

```
{
  "success": true,
  "message": "Group created successfully",
  "data": {
    "group_id": 10,
    "group_title": "Engineering",
    "group_type": "group",
    "admin_user_id": 1,
    "is_active": true,
    "created_at": "...",
    "updated_at": "..."
  }
}
```

Side effects: `GroupUserMap` bulk insert: **unique** union of `[admin, ...user_ids]`; admin row has `is_admin: true`.

Errors

- **400** — validation.
- **401** — missing `x-user-id`.

`PUT /api/v1/groups/:group_id` — Update group title (admin only)

Body validation: `updateGroupSchema` (`group_title` 2–120).

Request body

```
{
  "group_title": "Engineering Team"
}
```

Success — 200 — returns updated `ChatGroup` JSON.

Errors

- **403** — caller not group admin (`HttpError`).

- **404** — group not found or inactive.
- **401** — missing `x-user-id`.

`GET /api/v1/groups/:group_id/members` — List active members

Success — 200

```
{
  "success": true,
  "message": "Group members fetched successfully",
  "data": [
    {
      "user_id": 1,
      "first_name": "Ada",
      "last_name": "Lovelace",
      "email_id": "ada@example.com",
      "is_active": true,
      "created_at": "...",
      "updated_at": "..."
    }
  ]
}
```

Logic: active memberships (`is_exited: false`) → **`User`** with **`is_active: true`**.
Note: No membership check for caller; any client with `x-user-id` can list members if they know `group_id`.

`POST /api/v1/groups/:group_id/members` — Add members (admin only)

Body validation: `addGroupMembersSchema` — `user_ids` non-empty array of positive integers.

Request body

```
{
  "user_ids": [4, 5]
}
```

Success — 200

```
{
  "success": true,
  "message": "Members added successfully"
}
```

Logic: Skips users already mapped with `is_exited: false`. Only inserts missing IDs.

Errors

- **403** / **404** — same patterns as admin operations.

`DELETE /api/v1/groups/:group_id/members/:user_id` **— Soft-remove member (admin only)**

Success — 200

```
{
  "success": true,
  "message": "Member removed successfully"
}
```

Errors

- **400** — admin attempts to remove self.
- **404** — mapping not found / already exited.
- **403** — not admin.

Side effects: Sets `is_exited: true`, `exited_at` = now.

`POST /api/v1/groups/:group_id/admin/:user_id` **— Transfer admin**

Success — 200 — returns updated `ChatGroup` (new `admin_user_id`).

Errors

- **404** — target user not an active member; group not found.
- **403** — caller not current admin.

Side effects: Clears `is_admin` on old admin's map row; sets `is_admin` on new admin; updates `ChatGroup.admin_user_id`.

10.3 Chat (`/api/v1/chat`)

All routes require `x-user-id`.

`GET /api/v1/chat/groups/:group_id/messages` **— Group message history**

Success — 200

```
{
  "success": true,
  "message": "Group messages fetched successfully",
```

```
  "data": [
    {
      "message_id": 100,
      "group_id": 10,
      "sender_user_id": 1,
      "receiver_user_id": null,
      "message_text": "Hello team",
      "message_type": "text",
      "sent_at": "..."
    }
  ]
}
```

Rules

- Caller must be active group member (**403** if not).
- **Max 500** messages, ordered `sent_at` **ASC**.

`POST /api/v1/chat/groups/:group_id/messages` **— Send group message (REST)**

Body validation: `sendMessageSchema` — `message_text` 1–2000 chars.

Request body

```
{
  "message_text": "Hello team"
}
```

Success — 201

```
{
  "success": true,
  "message": "Group message sent successfully",
  "data": {
    "message_id": 101,
    "group_id": 10,
    "sender_user_id": 1,
    "receiver_user_id": null,
    "message_text": "Hello team",
    "message_type": "text",
    "sent_at": "..."
  }
}
```

Side effects

1. Persists `ChatMessage` (group).

2. **Notifications:** for each other member, creates `Notification` (`type: "group"`, title/body truncated body preview up to 120 chars).
3. **Socket.IO:** `io.to("group:" + group_id).emit("group:message", message)`.

Errors

- **403** — not a group member.

`GET /api/v1/chat/groups/:group_id/online` — Online users in group (presence)

Success — 200

```
{
  "success": true,
  "message": "Group online users fetched successfully",
  "data": {
    "group_id": 10,
    "online_count": 3,
    "online_user_ids": [1, 2, 5]
  }
}
```

Rules: Caller must be group member (**403**).

Semantics: `online_user_ids` = users who have joined the group room in `presenceService` (Socket connection + group join on connect). Not necessarily identical to “has socket open” for users in multiple groups.

`GET /api/v1/chat/direct/:peer_user_id/online`

Success — 200

```
{
  "success": true,
  "message": "Direct user online status fetched successfully",
  "data": {
    "user_id": 2,
    "is_online": true
  }
}
```

Note: No check that a “direct relationship” exists.

`GET /api/v1/chat/direct/:peer_user_id/messages` — DM history

Success — 200

```
{
  "success": true,
  "message": "Direct messages fetched successfully",
  "data": [
    {
      "message_id": 200,
      "group_id": null,
      "sender_user_id": 1,
      "receiver_user_id": 2,
      "message_text": "Hi",
      "message_type": "text",
      "sent_at": "..."
    }
  ]
}
```

Query logic: `group_id` is null, and (sender, receiver) pair matches **(auth, peer)** or **(peer, auth)**. **Limit 500**, `sent_at` **ASC**.

Note: No authorization beyond knowing peer id + having `x-user-id`.

`POST /api/v1/chat/direct/:peer_user_id/messages` **— Send DM (REST)**

Body: same as group send (`message_text` 1–2000).

Success — 201 — created `ChatMessage` with `group_id: null`, `receiver_user_id` = peer.

Side effects

1. **Notification** for peer: `notification_type: "direct"`, `related_user_id` = sender.
2. **Socket.IO:** emits `direct:message` to `user:<peer>` and `user:<sender>`.

`POST /api/v1/chat/messages/:message_id/seen` **— Mark message seen**

Success — 200

```
{
  "success": true,
  "message": "Message marked as seen",
  "data": {
    "message_seen_id": 50,
    "message_id": 200,
    "user_id": 2,
    "seen_at": "..."
  }
}
```

Logic: `findOrCreate` on **`MessageSeen`** for `(message_id, user_id)`.

Socket.IO: **`io.emit("message:seen", { message_id, seen_users })`** — **broadcast to all** connected clients (not scoped to group/user room).

Errors

- **404** — message id does not exist.

Note: No verification that the user is allowed to see that message (e.g. member of group or DM participant).

10.4 Notifications (`/api/v1/notifications`)

All routes require **`x-user-id`**.

`GET /api/v1/notifications` — **Paginated list**

Query	Values	Default
`page`	integer ≥ 1	1
`limit`	1–50	10
`status`	`all` | `read` | `unread`	`all`

Example: `GET /api/v1/notifications?page=1&limit=20&status=unread`

Success — 200

```
{
  "success": true,
  "message": "Notifications fetched successfully",
  "data": {
    "items": [
      {
        "notification_id": 1,
        "user_id": 2,
        "notification_type": "direct",
        "notification_title": "New direct message",
        "notification_body": "Hello…",
        "is_read": false,
        "group_id": null,
        "related_user_id": 1,
        "created_at": "..."
      }
    ],
    "total": 15,
    "unread_count": 3,
```

```
      "page": 1,
      "limit": 20,
      "total_pages": 1,
      "status": "unread"
    }
  }
```

Notes

- `unread_count` is always the count of unread for the user, independent of `status` filter.
- `total` / pagination reflect the filtered list.

`POST /api/v1/notifications/:notification_id/read`

Success — 200 — returns updated notification with `is_read: true`.

Errors

- **404** — notification not found for this user.

---

11. Data model (Sequelize / MySQL)

11.1 ER overview (logical)

- `user` — identity profile.
- `chat_group` — group metadata; `admin_user_id` → `user`.
- `group_user_map` — many-to-many **user ↔ group** with soft exit (`is_exited`, `exited_at`), `is_admin` flag.
- `chat_message` — either **group** (`group_id` set, `receiver_user_id` null) or **direct** (`group_id` null, `receiver_user_id` set).
- `message_seen` — read receipt per `(message_id, user_id)` unique.
- `notification` — per-user inbox row; optional `group_id` / `related_user_id`.

Associations are declared in `src/models/index.ts` (`belongsTo` only; no `hasMany` in file but FKs support queries).

11.2 Table: `user`

Column	Type (logical)	Constraints
`user_id`	BIGINT UNSIGNED	PK, AI
`first_name`	VARCHAR(100)	NOT NULL
`last_name`	VARCHAR(100)	NOT NULL
`email_id`	VARCHAR(200)	NOT NULL, UNIQUE `uk_user_email_id`
`is_active`	BOOLEAN	NOT NULL, default true

Column	Type (logical)	Constraints
`created_at`	DATE	NOT NULL, default NOW
`updated_at`	DATE	NOT NULL, default NOW

Indexes: `email_id`, `is_active`.
Sequelize: `timestamps: false` (manual columns).

11.3 Table: `chat_group`

Column	Type	Constraints
`group_id`	BIGINT UNSIGNED	PK, AI
`group_title`	VARCHAR(120)	NOT NULL
`group_type`	ENUM(`group`)	NOT NULL, default `group`
`admin_user_id`	BIGINT UNSIGNED	NOT NULL
`is_active`	BOOLEAN	NOT NULL, default true
`created_at` / `updated_at`	DATE	NOT NULL

Index: `admin_user_id`.

11.4 Table: `group_user_map`

Column	Type	Constraints
`group_user_map_id`	BIGINT UNSIGNED	PK, AI
`group_id`	BIGINT UNSIGNED	NOT NULL
`user_id`	BIGINT UNSIGNED	NOT NULL
`is_admin`	BOOLEAN	NOT NULL, default false
`joined_at`	DATE	NOT NULL, default NOW
`exited_at`	DATE	NULL
`is_exited`	BOOLEAN	NOT NULL, default false

Indexes: `group_id`, `user_id`, UNIQUE(`group_id`, `user_id`).

11.5 Table: `chat_message`

Column	Type	Constraints
`message_id`	BIGINT UNSIGNED	PK, AI
`group_id`	BIGINT UNSIGNED	NULL (NULL = direct)
`sender_user_id`	BIGINT UNSIGNED	NOT NULL

Column	Type	Constraints
receiver_user_id	BIGINT UNSIGNED	NULL
message_text	TEXT	NOT NULL
message_type	ENUM(text)	NOT NULL, default text
sent_at	DATE	NOT NULL, default NOW

Indexes: group_id, sender_user_id, receiver_user_id, sent_at.

11.6 Table: message_seen

Column	Type	Constraints
message_seen_id	BIGINT UNSIGNED	PK, AI
message_id	BIGINT UNSIGNED	NOT NULL
user_id	BIGINT UNSIGNED	NOT NULL
seen_at	DATE	NOT NULL, default NOW

UNIQUE(message_id, user_id).

11.7 Table: notification

Column	Type	Constraints
notification_id	BIGINT UNSIGNED	PK, AI
user_id	BIGINT UNSIGNED	NOT NULL
notification_type	ENUM(group,direct,system)	NOT NULL, default system
notification_title	VARCHAR(150)	NOT NULL
notification_body	VARCHAR(600)	NOT NULL
is_read	BOOLEAN	NOT NULL, default false
group_id	BIGINT UNSIGNED	NULL
related_user_id	BIGINT UNSIGNED	NULL
created_at	DATE	NOT NULL, default NOW

Indexes: user_id, is_read.

---

12. Service layer

Service	Responsibility

Service	Responsibility
`UserService`	Create user, list all users, paginated directory with online flags via `presenceService`.
`GroupService`	Group CRUD-ish operations, membership, admin checks, soft exit, admin transfer, `ensureMembership`, `getGroupAsAdmin`.
`ChatService`	Persist/list messages (group + direct), `markSeen`, `getMessageSeenUsers`, `listGroupMemberIds`.
`NotificationService`	Create notification rows; paginated list with read/unread/all; mark read.
`presenceService`	In-memory maps: user ↔ socket set; group ↔ online user set; join/leave on socket lifecycle.

Dependency direction: Controllers → Services → Models. `ChatController` also calls `io` and `NotificationService`.

---

13. Validators (Joi)

Schema	File	Used by
`createUserSchema`	`user.validator.ts`	`POST /users`
`createGroupSchema`, `updateGroupSchema`, `addGroupMembersSchema`	`group.validator.ts`	Group create/update/add members
`sendMessageSchema`	`chat.validator.ts`	Group + direct REST send

---

14. Real-time design (Socket.IO)

14.1 Server singleton (`src/socket/io.ts`)

- `io` is assigned after server start via `setSocketServer`. Any code importing `io` before start would see `undefined` — current usage from HTTP controllers runs after listen, so `io` is set.

14.2 Connection auth (`register-events.ts`)

- `socket.handshake.auth.user_id` must be a positive finite number; otherwise **immediate** `disconnect`.

14.3 Rooms

Room name	Members
`user:<user_id>`	Sockets that authenticated as that user
`group:<group_id>`	Sockets for users who were in `listMyGroups(user_id)` at **connection time**

Caveat: If group membership changes while connected, room membership is **not** refreshed until reconnect.

14.4 Presence updates

- On connect: `user:presence` `{ user_id, is_online: true }` **broadcast**; per group `group:presence` `{ group_id, online_cnt }` to group room.
- On disconnect: `user:presence` with `is_online` from `presenceService.isUserOnline` (false if last tab closed); per group `group:presence` with updated count.

14.5 Socket events (client → server)

Event	Payload	Behavior
`chat:group:send`	`{ group_id: number, message_text: string }`	`ChatService.sendGroupMessage`; `group:message` to `group:<group_id>`. **No notification rows.**
`chat:direct:send`	`{ receiver_user_id: number, message_text: string }`	`ChatService.sendDirectMessage`; `direct:message` to sender + receiver user rooms. **No notification rows.**

14.6 Server → client events (emitted from code)

Event	When
`group:message`	After group send (REST or socket)
`direct:message`	After DM send (REST or socket)
`message:seen`	After REST mark seen — payload `{ message_id, seen_users }`
`user:presence`	Connect / disconnect
`group:presence`	Connect / disconnect per group

---

15. Behavioral differences: REST vs Socket

Concern	REST (`ChatController`)	Socket (`register-events`)
Notifications	Created for group (all except sender) and DM (receiver)	**Not created**
DM emit	Same as socket	Same
Group emit	Same	Same
Validation	Joi on body	**No Joi** — trust client payload types

Clients should treat REST as the path that maintains the **notification inbox**.

---

16. Concurrency, scale, and ops notes

- **Single Node process** — `presenceService` state is **local memory**; horizontal scaling would require a shared adapter (Redis) not present here.
- **`sequelize.sync(alter)`** in development can apply destructive DDL — avoid pointing at production shared DBs.
- **MySQL socket** — default `socketPath` in `dialectOptions`; override with env for TCP-only hosts.
- **CORS `*` + open `x-user-id`** — not suitable for hostile networks without additional controls.

---

17. File-to-concern map

Path	Role
`src/app.ts`	Express app: middleware + `/api/v1` + 404 + error handler
`src/server.ts`	DB, sync, HTTP+Socket.IO, shutdown
`src/config/env.ts`	Environment
`src/config/database.ts`	Sequelize instance
`src/routes/index.ts`	API mount order + auth gate
`src/routes/*.routes.ts`	Route tables
`src/controllers/*.ts`	HTTP orchestration
`src/services/*.ts`	Business logic + DB
`src/models/*.ts`	Sequelize models + `models/index.ts` associations
`src/middlewares/*.ts`	Auth context, validation, errors, 404
`src/validators/*.ts`	Joi schemas
`src/utils/*.ts`	Response helpers, `HttpError`, `asyncHandler`
`src/socket/io.ts`	Socket.IO singleton export
`src/socket/register-events.ts`	Connection handlers + socket chat
`src/types/express-request-augmentation.ts`	`Request` typing